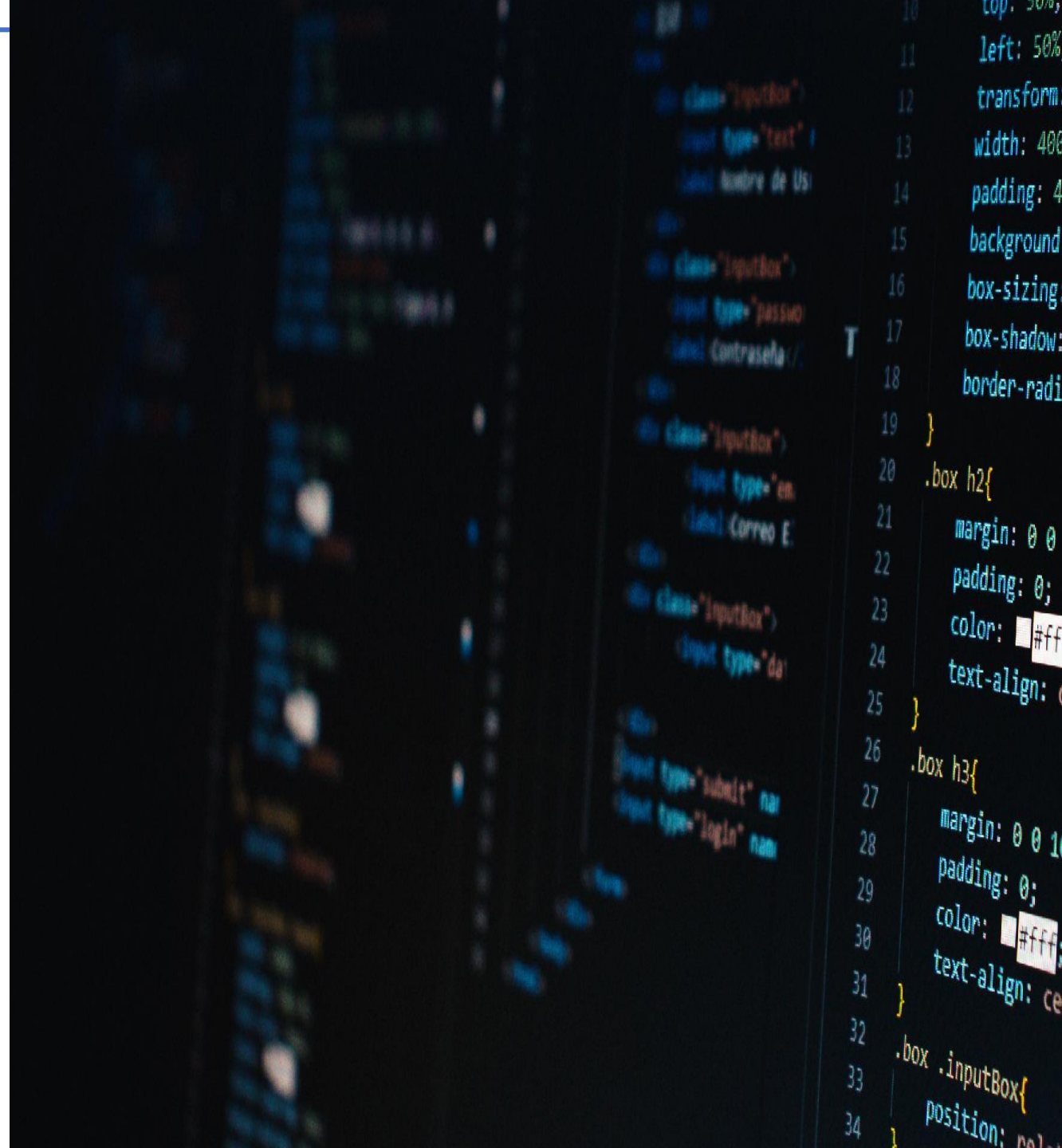
Bahan Ajar PBO

Bab 3

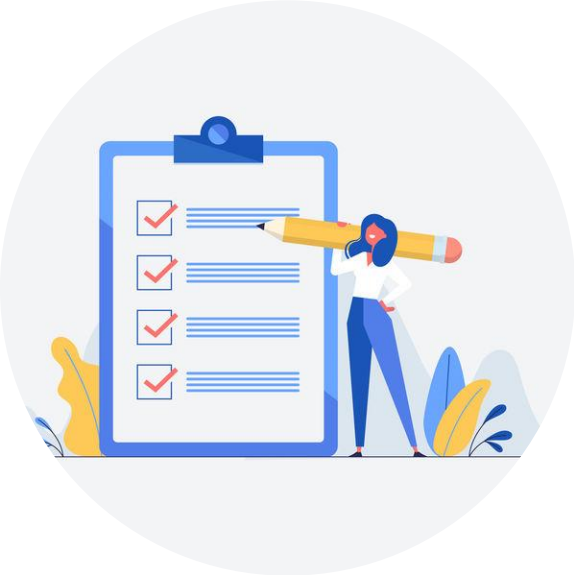
Eksepsi

Oleh:
Abidatul Izzah, M.Kom
Fikha Rizky Aullia, M.Pd



DIII Manajemen Informatika, Kampus Kediri
POLITEKNIK NEGERI MALANG

Capaian Pembelajaran



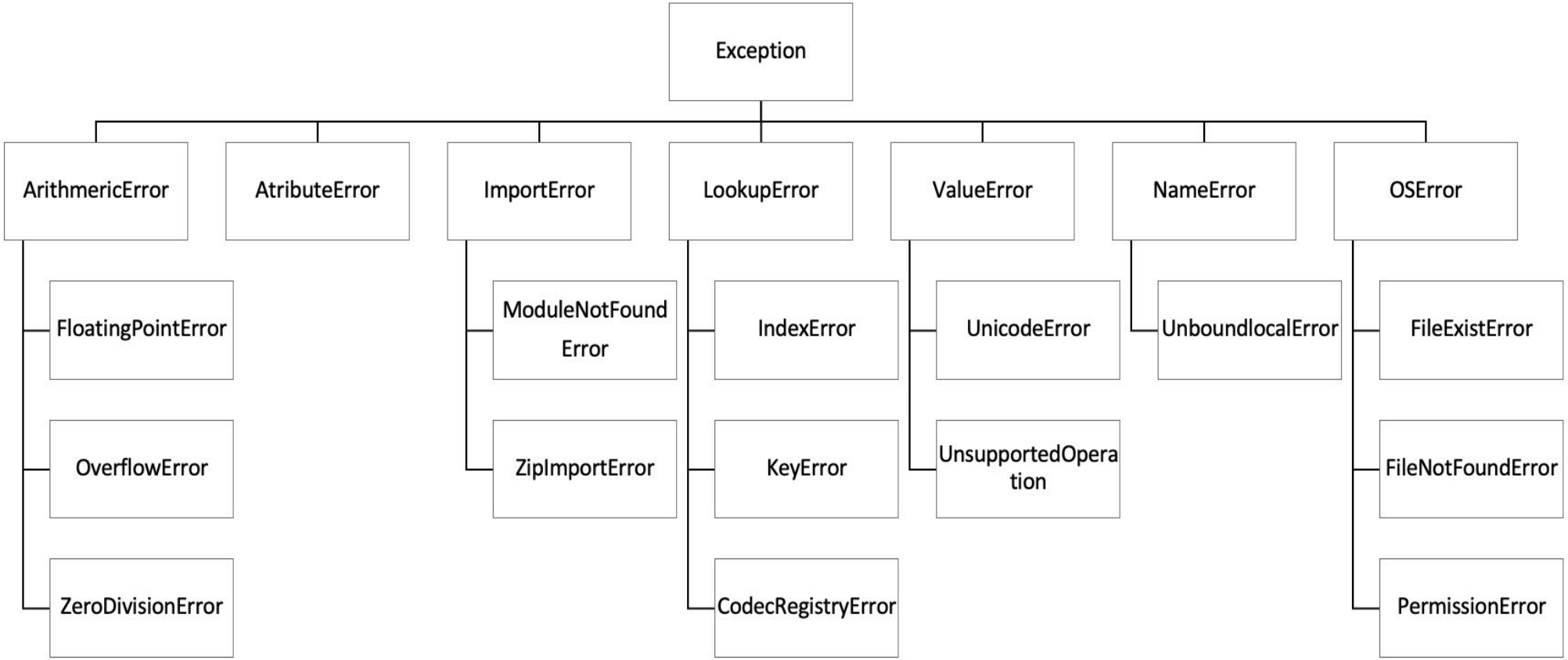
Setelah menempuh bab ini, diharapkan mahasiswa mampu menguasai kompetensi berikut:

1. Mampu menjelaskan konsep eksepsi dalam python
2. Mampu mengimplementasikan eksepsi dalam sebuah kasus program sederhan

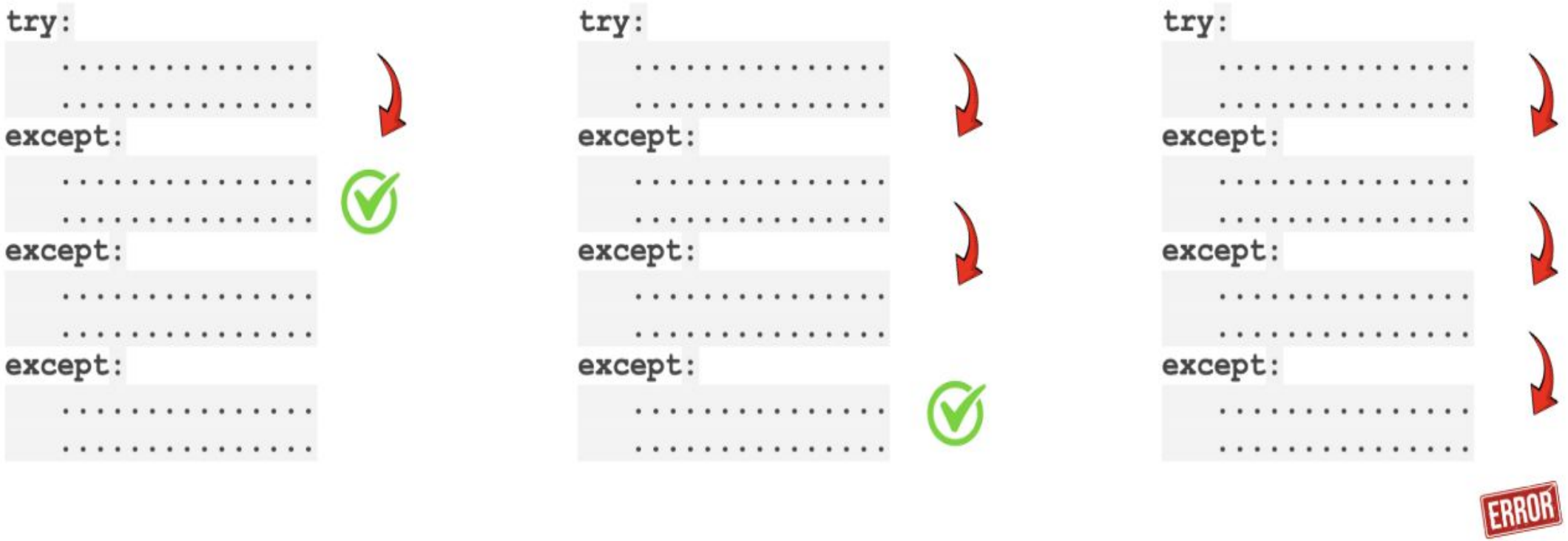
Mengenal Error Handling

- Kesalahan (*error*) pada pemrograman hampir pasti dialami. Namun perlu diketahui bahwa *error* tidak hanya terjadi karena kesalahan syntax, namun juga dapat terjadi karena operasi matematika, logika, maupun kesalahan indeks.
- Error menyebabkan program yang dibangun menjadi **terhenti dan tidak dapat dilanjutkan.**
- Oleh karena itu python memiliki cara untuk mengatasi error (error handling) dengan menyediakan **kelas Eksepsi (*exception*)**.

Mengenal Error Handling pada Python

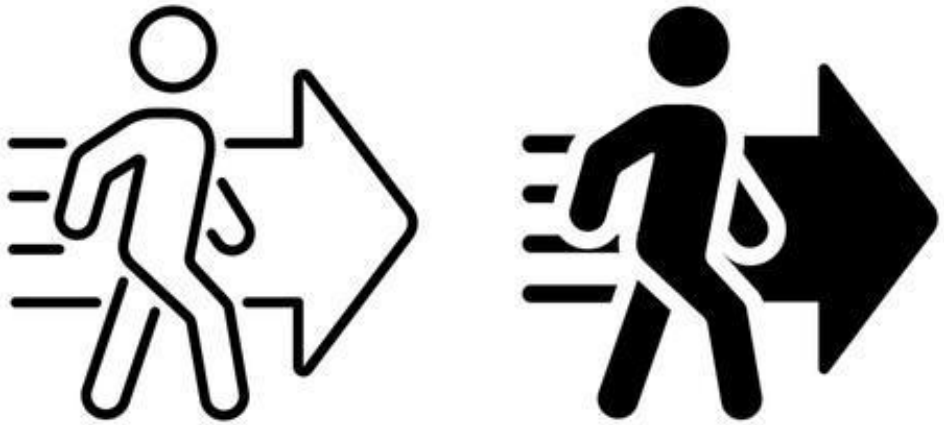


Struktur Error Handling pada Python



With or without Error Handling

Pendekatan	Kelebihan	Kekurangan
Tanpa error handling	Kode sederhana	Program mudah berhenti
try-except umum	Mudah dipahami	Error tidak spesifik
Exception spesifik	Debugging lebih mudah	Harus memahami jenis exception
Multiple exception	Menangani berbagai error	Struktur lebih panjang



Praktikum 3.1
Praktikum 3.2
Praktikum 3.3
Praktikum 3.4
Praktikum 3.5

Praktikum 3.1

```
bilangan1 = 1
bilangan2 = 0
print(bilangan1/bilangan2)
print("<< End Program >>")
```

```
if bilangan2 != 0:
    print(bilangan1/bilangan2)
else:
    print("Operasi matematika tidak       dijalankan")
print("<< End Program >>")
```

```
my_list = []
idx = 0
print(my_list[idx])
print("<< End Program >>")
```

```
my_list = []
idx = 0

if (idx>-1 & idx<len(my_list)):
    print(my_list[idx])
else:
    print("Operasi tidak dapat dijalankan")
print("<< End Program >>")
```

```
bilangan1 = 1
bilangan2 = 0

try:
    print(bilangan1/bilangan2)
except:
    print("Operasi matematika tidak       dijalankan")
print("<< End Program >>")
```

```
my_list = [9]
idx = 0

try:
    print(my_list[idx])
except:
    print("Operasi tidak dapat dijalankan")
print("<< End Program >>")
```

Praktikum 3.2

```
def print_exception_tree(thisclass, nest = 0):
    if nest > 1:
        print("    |" * (nest - 1), end="")
    if nest > 0:
        print("    +---", end="")

    print(thisclass.__name__)

    for subclass in thisclass.__subclasses__():
        print_exception_tree(subclass, nest + 1)
```

```
print_exception_tree(BaseException)
```

```
fikhaaullia@MacBook-Air-Fikha Bab 3 % /usr
BaseException
    +---Exception
    |    +---TypeError
    |    +---StopAsyncIteration
    |    +---StopIteration
    |    +---ImportError
    |    |    +---ModuleNotFoundError
    |    |    +---ZipImportError
    |    +---OSError
    |    |    +---ConnectionError
    |    |    |    +---BrokenPipeError
    |    |    |    +---ConnectionAbortedError
    |    |    |    +---ConnectionRefusedError
    |    |    |    +---ConnectionResetError
    |    |    +---BlockingIOError
    |    |    +---ChildProcessError
    |    |    +---FileExistsError
    |    |    +---FileNotFoundError
    |    |    +---IsADirectoryError
    |    |    +---NotADirectoryError
    |    |    +---InterruptedError
    |    |    +---PermissionError
    |    |    +---ProcessLookupError
    |    |    +---TimeoutError
    |    |    +---UnsupportedOperation
    |    |    +---ItimerError
    |    +---EOFError
    |    +---RuntimeError
    |    |    +---RecursionError
    |    |    +---NotImplementedError
    |    |    +---_DeadlockError
    |    +---NameError
    |    |    +---UnboundLocalError
    |    +---AttributeError
    |    +---SyntaxError
    |    |    +---IndentationError
    |    |    |    +---TabError
    |    +---LookupError
    |    |    +---IndexError
    |    |    +---KeyError
    |    |    +---CodecRegistryError
    |    +---ValueError
    |    |    +---UnicodeError
    |    |    |    +---UnicodeEncodeError
    |    |    |    +---UnicodeDecodeError
    |    |    |    +---UnicodeTranslateError
    |    |    +---UnsupportedOperation
    |    +---AssertionError
    |    +---ArithmeticError
    |    |    +---FloatingPointError
    |    |    +---OverflowError
    |    |    +---ZeroDivisionError
    |    +---SystemError
    |    |    +---CodecRegistryError
    |    +---ReferenceError
    |    +---MemoryError
    |    +---BufferError
    |    +---Warning
    |    |    +---UserWarning
    |    |    +---DeprecationWarning
    |    |    +---PendingDeprecationWarning
    |    |    +---SyntaxWarning
    |    |    +---RuntimeWarning
    |    |    +---FutureWarning
    |    |    +---ImportWarning
    |    |    +---UnicodeWarning
    |    |    +---BytesWarning
    |    |    +---ResourceWarning
    |    +---Error
    +---GeneratorExit
    +---SystemExit
    +---KeyboardInterrupt
```

Praktikum 3.3

```
bil1=9
bil2=0
x=bil1/bil2
print("<< End Program >>")
```

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
import math

print(math.a)
print("<< End Program >>")
```

```
try:
    print(math.a)
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
from time import datetime
print("<< End Program >>")
```

```
try:
    from time import datetime
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
import mymodul
print("<< End Program >>")
```

```
try:
    import mymodul
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
list = [1, 2, 3]
print(list[3])
print("<< End Program >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except Exception as e:
    print(e)
print("<< End Program >>")
```

Praktikum 3.3

```
harga = {"apel": 25000, "jeruk": 20000, "mangga": 15000}
harga["anggur"]
print("<< End Program >>")
```

```
try:
    harga = {"apel": 25000, "jeruk": 20000, "mangga": 15000}
    harga["anggur"]
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
list=[]
x=max(list)
print("<< End          >>")
```

```
try:
    list=[]
    x=max(list)
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
text='Hallo, mari belajar eksepsi'
print(teks)
print("<< End Program >>")
```

```
try:
    text='Hallo, mari belajar eksepsi'
    print(teks)
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
varString = "Hello"
varInt = 100
print(varString + varInt)
print("<< End Program >>")
```

```
try:
    varString = "Hello"
    varInt = 100
    print(varString + varInt)
except Exception as e:
    print(e)
print("<< End Program >>")
```

```
with open("sample.txt", mode="r") as file:
    print(file.read())
print("<< End Program >>")
```

```
try:
    with open("sample.txt", mode="r") as file:
        print(file.read())
except Exception as e:
    print(e)
print("<< End Program >>")
```

Praktikum 3.4

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except ZeroDivisionError as e:
    print(e)
print("<< End Program >>")
```

```
import math

try:
    print(math.a)
except AttributeError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    from time import datetime
except ImportError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    import mymodul
except ModuleNotFoundError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except IndexError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    harga = {"apel": 25000, "jeruk": 20000, "mangga":
15000}
    harga["anggur"]
except KeyError as e:
    print(e)
print("<< End Program >>")
```

Praktikum 3.4

```
try:
    list=[]
    x=max(list)
except ValueError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    varString = "Hello"
    varInt = 100
    print(varString + varInt)
except TypeError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    text='Hallo, mari belajar eksepsi'
    print(teks)
except NameError as e:
    print(e)
print("<< End Program >>")
```

```
try:
    with open("sample.txt", mode="r") as file:
     print(file.read())
except Exception as e:
    print(e)
print("<< End Program >>")
```

Praktikum 3.5

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except IndexError:
    print("Ada kesalahan indeks")
```

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except IndexError:
    print("Ada kesalahan indeks")
except ZeroDivisionError:
    print("Ada kesalahan pembagian 0")

print("<<              >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except ZeroDivisionError:
    print("Operasi tidak bisa dijalankan")

print("<<              >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except ZeroDivisionError:
    print("Operasi tidak bisa dijalankan")
except IndexError:
    print("Ada kesalahan indeks")
print("<< End Program >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except ZeroDivisionError:
    print("Operasi tidak bisa dijalankan")
except:
    print("Ada kesalahan ")
print("<< End Program >>")
```

```
try:
    list = [1, 2, 3]
    print(list[3])
except LookupError:
    print("Ada kesalahan")
except IndexError:
    print("Ada kesalahan indeks")
except:
    print("Ada kesalahan ")
print("<< End Program >>")
```

Praktikum 3.5

```
try:
    list = [1, 2, 3]
    print(list[3])
except IndexError:
    print("Ada kesalahan indeks")
except LookupError:
    print("Ada kesalahan")
except:
    print("Ada kesalahan ")
print("<< End Program >>")
```

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except ArithmeticError:
    print("Ada kesalahan aritmatika")
except FloatingPointError:
    print("Ada kesalahan floating point")
except ZeroDivisionError:
    print("Ada pembagian dengan 0")
print("<< End Program >>")
```

```
try:
    bil1=9
    bil2=0
    x=bil1/bil2
except FloatingPointError:
    print("Ada kesalahan floating point")
except ZeroDivisionError:
    print("Ada pembagian dengan 0")
except ArithmeticError:
    print("Ada kesalahan aritmatika")
print("<< End Program >>")
```

Latihan

Buat skenario kesalahan berikut ini dalam sebuah program dan tangkap kesalahan tersebut menggunakan kelas eksepsi yang sesuai. Buat kelas eksekusi yang bertahap dari khusus ke baseException.

- OverflowError > ArithmeticError > Exception
- FileExistsError > OSError > Exception
- ZipImportError > ImportError > Exception

Latihan

Buat program sederhana ATM dengan fitur tarik tunai yang mampu menangani kesalahan untuk kondisi berikut:

- Nominal yang dimasukkan bukan angka
- Saldo tidak mencukupi
- Nominal kurang dari atau sama dengan 0

Gunakan exception untuk menangani seluruh error tersebut.